

Fix layout mappings all static extent default constructor

Document #: P2798R0
Date: 2023-02-07
Project: Programming Language C++
LWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- [1 Revision History](#)
 - [1.1 Initial Version 2023-02 Mailing](#)
- [2 Rationale](#)
 - [2.1 Proposed Wording](#)

§ 1 Revision History

§ 1.1 Initial Version 2023-02 Mailing

§ 2 Rationale

When default constructing a `layout_left`, `layout_right` or `layout_stride` mapping with all static extents, there is currently no precondition check guarding against overflow of the `index_type`.

A simple example is constructing the following `layout_left` mapping:

```
constexpr size_t N = 4'000'000;  
layout_left::mapping<extents<int, N, N>> map;
```

Each individual extent fits into `int` but the theoretical required span size does not.

The non-default constructors already deal with this issue via a preconditions such as:

Preconditions: The size of the multidimensional index space `e` is representable as a value of type `index_type` ([basic.fundamental]).

For cases where at least one of the extents is dynamic, the default constructor is also fine, since it will result in an empty index space. That leaves only the case for all static extents to be fixed. We propose here to add a mandate to the mapping classes of `layout_left`, `layout_right`, and `layout_stride` that the forward product of extents is representable as `index_type`.

§ 2.1 Proposed Wording

In subsection 24.7.3.4.5.1 [mdspan.layout.left.overview] add after paragraph 3:

Mandates: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

In subsection 24.7.3.4.6.1 [mdspan.layout.right.overview] add after paragraph 3:

Mandates: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

In subsection 24.7.3.4.7.1 [mdspan.layout.stride.overview] add after paragraph 3:

Mandates: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.